Documento di Progetto

Titolo: Gioco degli Scacchi

Descrizione:
L'obiettivo di questo programma è di avvicinare chiunque al gioco degli scacchi; pertanto, si tratterà di un software didattico che permetterà a qualsiasi utente di imparare quali sono le mosse e le regole basilari, come per esempio quella di portare al termine la partita una volta iniziata, a meno che non si chieda: patta o abbandono.

Questo gioco permetterà all'utente di *registrarsi e/o accedere* con l'immissione di nickname e password, verrà chiesto ad uno dei due utenti con il quale vuole giocare, dopo sarà possibile scegliere un'operazione dal menu principale tra:

- *New* - per iniziare una nuova partita;
- *Classifica* - per stampare a schermo il punteggio dei giocatori registrati, le partite giocate e l'elo (punteggio che rappresenta l'abilita e il livello del giocatore);
- *Help* - per leggere le istruzioni del gioco;
- *Exit* - per uscire dal gioco.

Selezionando *New* oltre a poter iniziare la partita si accede ad un menu secondario che permetterà:

- *Comandi* – per visualizzare i comandi possibili;
- *Classifica* - per stampare a schermo il punteggio dei giocatori*;*
- *Abbandono* - per abbandonare la partita, in questo caso la partita è persa;
- Patta - per finire la partita in pareggio.

Appena finisco una partita stampa le mosse giocate con l'annotazione specifica degli scacchi, e la classifica con i dati aggiornati.
Nello stampare la scacchiera si utilizzerà la codifica Unicode al fine di visualizzare i caratteri speciali che rappresentato i pezzi con i rispettivi colori.

INTERFACCIA UTENTE:

Utilizzo la funzione **caricaGiocatori**() per caricare i giocatori già rigistrati.

Chiamo la funzione **Login** (), per controllare se esiste l'username e controllare se la sua password è quella inserita

Chiamo la funzione **Registrazione** (), per inserire un username non esistente password ed ELO, e poi **inserisciGiocatore**().

```
SCHACCHI

accedi
registrati

Digita 'accedi' se ti sei già registrato o 'registrati' se sei un nuovo utente:
```

```
accedi
registrati

Digita 'accedi' se ti sei già registrato o 'registrati' se sei un nuovo utente:
accedi

Inserisci username:emily

Inserisci password:  marvel

Password corretta. Accesso in corso.

Benvenuto emily
```

```
accedi
registrati

Digita 'accedi' se ti sei già registrato o 'registrati' se sei un nuovo utente:
registrati

Inserisci username:  emanuele

Inserisci password (P.S. ricordala per poter accede altre volte):   casa

Inserisci il tuo ELO (se non hai mai giocato a scacchi e' 1440):  1440
```

```
emily vuoi giocare con il bianco o con il nero? (Digita 'bianco' o 'nero')
nero

new
classifica
help
exit
```

Exit per uscire dal gioco.

Chiedo ad uno dei due utenti appena entrati con quale colore voglia giocare.

```
emily vuoi giocare con il bianco o con il nero? (Digita 'bianco' o 'nero')
nero

new
classifica
help
exit
```

Per help stampo a video le regole del gioco degli scacchi e come si muovono i pezzi.

```
help

 Libretto Delle Istruzioni
 Le regole degli scacchi prevedono che ogni singolo pezzo si sposti sulla scacchiera in modo preciso.
 Il giocatore che muove per primo è colui che vede il proprio Re a destra della propria Regina e si chiama Bianco mentre l'avversario è il Nero.
 Apertura, mediogioco e finale sono le tre fasi che compongono ogni partita.
 Nessun pezzo, secondo il regolamento, può occupare una casella in cui si trova un altro pezzo dello stesso schieramento.
 Al contrario un pezzo può spingersi verso una casa occupata da un pezzo avversario eliminandolo e prendendo il suo posto sulla scacchiera.
 In questo caso si parla di "cattura" e si annota con x, mentre quando un pezzo "attacca" o "minaccia" una casa significa che può muoversi su di essa.
 Nelle partite ufficiali non si puo' annullare la mossa fatta.


 COME SI MUOVONO I PEZZI:
 ♙ Pedone: la prima mossa gli consente di muoversi di una o due case in avanti a patto che la casella finale e la casa saltata siano libere.
 Nelle mosse successive il pedone può avanzare di una sola casella.
 A differenza degli altri pezzi non può mai muoversi all'indietro e cattura gli avversari spostandosi in senso diagonale e sempre in avanti.
 ♘ Cavallo: può spostarsi e catturare sia su caselle nere che su caselle bianche e i suoi movimenti formano idealmente una "L".
 È l'unico dei sedici pezzi a cui è permesso "saltare" sia gli alleati che gli avversari e pur trovandosi dietro i pedoni può essere mosso senza spostar
li.
 ♗ Alfiere: può muoversi diagonalmente in base al numero delle caselle libere disponibili e la sua casa d'arrivo non può essere occupata da un pezzo am
ico.
 ♖ Torre: la torre può muoversi sia in senso verticale che orizzontale in base alle caselle libere disponibili davanti e di fianco.
 Se non ostacolata da uno qualunque dei pezzi, a prescindere dalla sua posizione, la torre controlla sempre 14 case.
 È protagonista, insieme al re, di una mossa particolare conosciuta con il nome di arrocco, l'unica che permette di spostare contemporaneamente due pezz
i e che consente al Re di muoversi di due caselle, l'arrocco può essere corto 0-0 o lungo 0-0-0, nel primo caso il re si muove alla destra del giocatore
, nel secondo caso alla sinistra.
 ♕ Regina: o donna (D), è il pezzo più forte e conosciuto in passato con il nome di "generale", "stratega" o "visir".
 Può spostarsi verticalmente, orizzontalmente o diagonalmente in base alle caselle libere disponibili e a ogni mossa può anche scegliere se muoversi com
e un alfiere o una torre.
 A differenza della torre non può effettuare la mossa dell'arrocco.
 ♔ Re: tra i sedici pezzi è il più importante e può muoversi in qualsiasi direzione ma una casa alla volta e a patto che la casella di destinazione non
 sia minacciata da un pezzo avversario.
 Al re è permesso catturare pezzi avversari muovendosi sulla casa occupata da questi ultimi e insieme alla torre può eseguire la mossa dell'arrocco.
 Quando il re è minacciato si trova sotto scacco.
 Lo scacco matto (o più semplicemente, matto) è invece una situazione che si verifica quando non più possibile per il giocatore difendere il re con altr
i pezzi o spostarlo in una casella libera che lo salverebbe dall'avversario.
 Il giocatore che utilizza questa mossa chiude la partita e si aggiudica la vittoria.
```

```
classifica

1 - emily pt=1.0 partite=1 elo=1440
2 - emanuele pt=0.0 partite=0 elo=1440
```

```
comandi
classifica
cronologia
patta
abbandono
```

Chiamo la funzione **stampa_classifica** (), per stampare tutti gli utenti ordinandoli per punteggio decrescente.
Quando finisce una partita richiamo **trova_e_sostituisci**() per reinserire il giocatore con il nuovo punteggio e elo ma prima **cancellaGiocatore**() per eliminare quest'ultimo con i vecchi dati.

Visualizzo un menu secondario, che permette di mostrare la classifica, i comandi, oltre a mostrare come terminare una partita: patta o abbandono.

Stampa, inoltre, la scacchiera indicando il colore che muove.

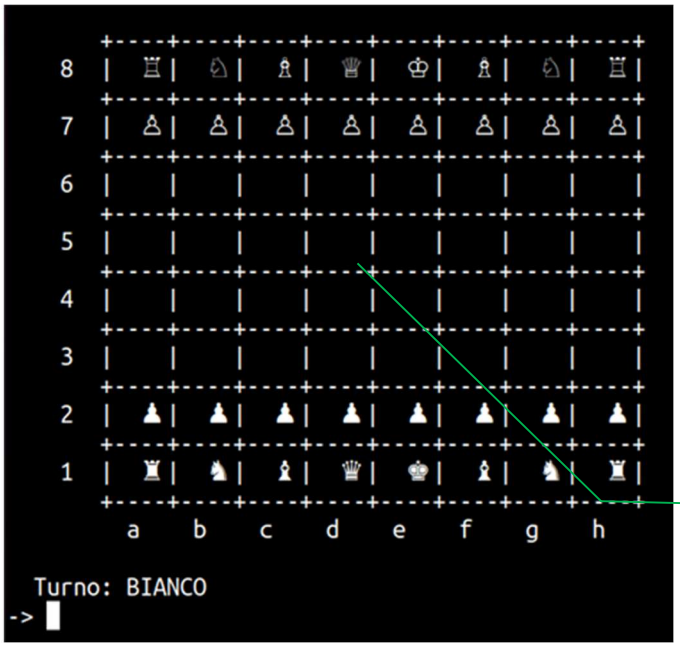
Dopo aver fatto una mossa richiamo la funzione **inserisci_storia**(), per salvare la mossa fatta, e quando scelgo cronologia richiamo **stampa**(). A fine programma **svuota_lista_crono**().

Mentre quando la partita finisce per matto, per patta o abbandono richiamo la funzione **svuota_lista_caselle()** , **svuota_lista_mosse()** e **svuota_lista_giocatori().**

Stampo a video i comandi possibili, e come deve essere scritta una mossa.

```
-> comandi
patta - la partita finisce in pareggio.
abbandono - mi arrendo (perdo la partita).
classifica - visualizza la classifica dei giocatori

es. mosse: inserisci le coordinate di partenza e arrivo: e2e4, e1g1 per arrocco
```



Dopo aver scelto new dal menu richiamo la funzione **inizializza_lista_mosse**() per inizializzare la liste delle mosse che si riempirà con la funzione **SalvaMossa**() i dati delle mosse nella funzione **GeneraMosse**(), mentre **inizializza**() per riempire la scacchiera con i pezzi. La funzione genera mosse genera le possibili mosse che ogni pezzo può fare, per esempio se il pedone parte dalla seconda riga per il bianco si può muovere di una o due caselle, e di conseguenza si origineranno due mosse una dove la casa di destinazione è 8 caselle in più e l'altra dove sarà +16, stesso ragionamento per tutti gli altri pezzi, invece per le mosse speciali come l'arrocco e la promozione del pedone vengono esaminati nella funzione input ogni volta. Dopo di che si stampa a video la scacchiera con la funzione **VisualizzaScacchiera**(). Inserisco la mossa e richiamo la funzione **Input**(), per analizzare se la mossa inserita è tra le possibili mosse in genera mosse, utilizzo **mossa_x_da**() e **mossa_x_a**() in una condizione dove questi ultimi se sono uguali, il primo (punto di partenza) e il secondo (punto di arrivo) della mossa appena inserita, muovo il pezzo, ovvero richiamo la funzione **trova_e_sostituisci_casella**(), per modificare il contenuto della casella di partenza (che diventa vuota) e quella d'arrivo (che prende il contenuto informativo della casella di partenza). La funzione **mossa_x_cat**() restituisce valore 0 se catturo 1, altrimenti, questo valore permette di scrivere un'annotazione scacchistica più precisa.
case_x_pezzo() e **case_x_colore**() sono due funzioni utilizzate per sapere il pezzo e il colore presenti in quella casella.
La funzione **InScacco**(), richiamata dopo aver fatto la mossa, trova la posizione del re e ritorna **CasaAttaccata**() che assume FALSE se il re è sotto scacco TRUE se non lo è.
Quando facciamo una mossa e il re è sotto scacco si annulla la mossa fatta e si controlla che non sia **Matto**() , la funzione matto genera le possibili mosse per chi muove e le fa una alla volta controllando che il re non sia sotto scacco, se ci sono mosse possibili si generano le mosse per l'avversario e se ci sono pezzi dell'avversario che attaccano la posizione del re allora è matto, altrimenti, si può continuare a giocare.

File scacchiera.h

```c
#ifndef CASELLA_H
#define CASELLA_H
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h>
#include <unistd.h>

/* descrizione di una casella della scacchiera */
typedef struct {
    int Colore;
    int Pezzo;
} Casella;

struct nodo_Casella
{
  Casella dato;
  struct nodo_Casella *next;
};

/* descrizione di una mossa identificatico della casa di partenza e di arrivo */
typedef struct {
    int Da;
    int A;
    int cat; //0 se catturo 1 altrimenti
} Mossa;

struct nodo_Mossa
{
  Mossa dato;
  struct nodo_Mossa *next;
};

struct nodo_Casella* Inizializza(struct nodo_Casella* head);
void trova_e_sostituisci_casella (struct nodo_Casella* head, int c, int p, int index);
int case_x_colore (struct nodo_Casella* head, int index);
int case_x_pezzo (struct nodo_Casella* head, int index);
int CasaAttaccata (struct nodo_Casella* head,int r, int ChiMuove);
void svuota_lista_casella (struct nodo_Casella** head);
void svuota_lista_mosse (struct nodo_Mossa** head);
void SalvaMossa(struct nodo_Mossa** head, int d, int a, int c);
void StampaMossa(struct nodo_Mossa* head,int NumeroMosse);
struct nodo_Mossa *GeneraMosse(struct nodo_Casella* head,int ChiMuove);
int InScacco(struct nodo_Casella* head, int ChiMuove);
void Matto(struct nodo_Casella* head, int ChiMuove);
int Input(struct nodo_Casella* head,char * Input, int ChiMuove);
void VisualizzaScacchiera(struct nodo_Casella* head, int ChiMuove);

#endif
```

File storia.h

```c
#ifndef ANNOTAZIONE_H
#define ANNOTAZIONE_H
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h>
#include <unistd.h>

/* informazioni per annotare una mossa */
struct nodo_Storia
{
    char Da_c;
    char Da_r;
    char A_c;
    char A_r;
    int cat;
    char l;
    char promozione;
    struct nodo_Storia *next;
};

/* OPERAZIONI */
void inserisci_storia (struct nodo_Storia** head,char* da_c,char* da_r, char* a_c, char* a_r, int c, char* let, char* prom);
void stampa(struct nodo_Storia** head,int matto);
void salvaAnnotazione(struct nodo_Storia** head,int matto, char* bianco, char* nero);
void svuota_lista_crono (struct nodo_Storia** head);

#endif
```

File giocatore.h

```c
#ifndef GIOCATORE_H
#define GIOCATORE_H
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h>
#include <unistd.h>

/* STRUTTURA DATI */
struct giocatore
{
    char username[20];
    char password[20];
    int elo;
    float punteggio;
    int partite;
    struct giocatore *next;
};

/* OPERAZIONI */
void inserisciGiocatore(struct giocatore **head,char username[],char password[],int ELO, float Pt, int partite);
void caricaGiocatori(struct giocatore **head);
void cancellaGiocatore(struct giocatore** head, char* nome);
void trova_e_sostituisci (struct giocatore** head, char* nomevinc, char* nomeperd, int patta);
char* Registrazione(struct giocatore **head);
char* Login(struct giocatore *head);
void salvaUtenti(struct giocatore* head);
void stampa_classifica(struct giocatore* head);
void svuota_lista_giocatori (struct giocatore** head);
#endif
```

Tipi di dato astratto

- nodo_Casella: lista che contiene il valore di una casella (pezzo e colore) e un puntatore next.
- nodo_Mossa: lista che contiene le informazioni di ogni mossa e un puntatore a next;
- nodo_Storia: lista che salva in coda tutte le mosse giocate.
- Giocatore: lista ordinata per punteggio dei giocatori.

Operazioni

scacchiera.h

- void trova_e_sostituisci_casella (struct nodo_Casella* head, int c, int p, int index): modifico pezzo e colore in una casella che si trova in una specifica posizione della lista che contiene 8*8 caselle numerate dalla più alta a sinistra alla più bassa a destra;
- int case_x_colore (struct nodo_Casella* head, int index): restituisce un intero pari a 0 se il contenuto della casella è un pezzo bianco, 1 se è nero, 6 se la casella non contiene alcun pezzo.
- Int case_x_pezzo (struct nodo_Casella* head, int index): restituisce un intero pari a 0 se nella casella c'è un pedone, 1 caallo, 2 alfiere, 3 torre, 4 regina, 5 re, 6 se la casella è vuota.
- Void svuota_lista_casella (struct nodo_Casella** head): svuoto la lista scacchiera.
- Void svuota_lista_mosse (struct nodo_Mossa* head): svuoto la lista delle mosse generate dalla funzione:" Genera mosse".
- void SalvaMossa (struct nodo_Mossa* head, int d, int a, int c): salva le mosse generate dalla funzione:" Genera mosse" in uno stack, avviene un inserimento in testa.
- struct nodo_Mossa *GeneraMosse (struct nodo_Mossa* head, int ChiMuove): presenta un if pezzo è uguale al pedone questo si potrà muovere di due caselle se parte dalla seconda riga, altrimenti si muoverà di una e di conseguenza verrà utilizzata la funzione salva mosse indicando es. pedone che si muove dalla casella e2 in quella e4 la casella di partenza verrà indicata con il numero 52 mentre quella di arrivo 36 poiché le caselle verranno numerate da 0 a 63. Per gli altri pezzi si utilizzerà una matrice costante che indica i movimenti che possono fare i vari pezzi.
- struct nodo_Casella* Inizializza (struct nodo_Casella* head): si utilizzerà un vettore costante che contiene l'ordine dei pezzi prima di iniziare il gioco e verrà caricata nella lista casella ad una determinata posizione il pezzo presente nel vettore.
- int CasaAttaccata (struct nodo_Casella* head, int r, int ChiMuove): controlla se un qualsiasi pezzo muovendosi possa avere la casella di destinazione coincidente con quella in cui si trova il re che chiameremo: K.
- int InScacco (struct nodo_Mossa* head, int ChiMuove): questa funzione trova la posizione del re di chi muove e restituisce e la funzione: casa attaccata.
- void Matto (struct nodo_Mossa* head int ChiMuove): questa funzione controlla se il re è sotto scacco e fa le mosse per il colore avversario per vedere se per ogni mossa che potrebbe fare il re continui ad essere sotto scacco, se continua ad essere attaccato è matto.
- void VisualizzaScacchiera (struct nodo_Casella* head, int ChiMuove): stampa la scacchiera con i pezzi con la codifica unicode.

storia.h

- void inserisci_storia (struct nodo_Storia** head,char* da_c,char* da_r, char* a_c, char* a_r, int c, char* let, char* prom): se la mossa inserita dall'utente è legale la inserisce in coda con davanti la lettera iniziale in maiuscolo del pezzo se questo è diverso da un pedone.
- void stampa(struct nodo_Storia** head,int matto): stampa la cronologia delle mosse fino a quando non finisce la lista, se C che rappresenta se un pezzo sta catturando viene stampata per es. e4xd5 altrimenti e2-e4.
- void svuota_lista_crono ((struct nodo_Storia** head): svuota la lista delle mosse giocate.
- void salvaAnnotazione(struct nodo_Storia** head,int matto, char* bianco, char* nero): si salvano le mosse giocate in un file.

giocatore.h

- void inserisciGiocatore (struct giocatore** headGiocatore, char username[], char password[], int ELO, float Pt, int partite): inserisce nella lista giocatori un utente in ordine in base al punteggio.
- void caricaGiocatori (struct giocatore** headGiocatore): carica un utente con tutti i suoi dati.
- void cancellaGiocatore (struct giocatore** headGiocatore, char* nome): questa funzione viene richiamata nella trova e sostituisci per eliminare il giocatore.
- void trova_e_sostituisci (struct giocatore** head, char* nomevincitore, char* nomeperdente): cerca il giocatore vincitore e in seguito il giocatore perdente e li cancella per poi reinserirli nella posizione corretta in base al punteggio modificato.
- char* Registrazione (struct giocatore** headGiocatore): chiede all'utente di inserire tutti i dati per la registrazione.
- char* Login (struct giocatore* headGiocatore): chiede all'utente di inserire username e password, se non esiste alcun utente registrato verrà registrato come nuovo utente.
- void stampa_classifica(struct giocatore* head): stampa la classifica indicando: nome, punteggio, Elo e partite giocate.
- void salvaUtenti(struct giocatore* head): si salva su file utenti la lista degli utenti registrati e i loro dati, e sul file classifica si salva la classifica in ordine di punteggio.
- void svuota_lista_giocatori (struct giocatore* head): svuota la lista giocatori.